

MODUL 4

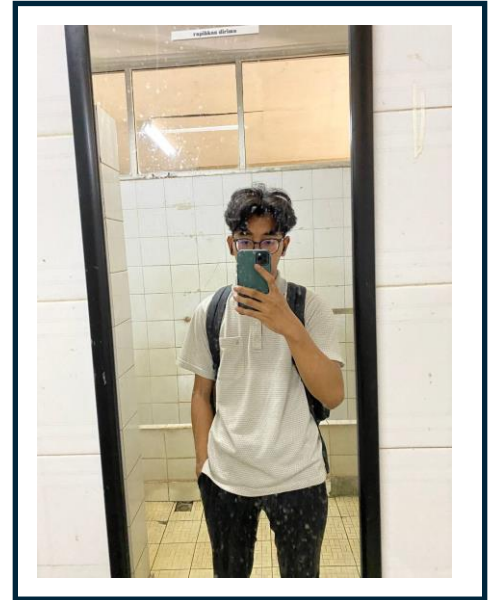
KOMPUTER GRAFIK 2D

TRANSFORMASI 2D (TRANSLASI, SCALE & BASIC ANIMASI)

D3 TEKNIK INFORMATIKA

JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA

POLITEKNIK NEGERI BANDUNG





CONTENTS

Konsep TRANSFORMASI 0

TRANSLASI..... 0

SCALING..... 2

ROTASI..... 3

HOMOGENEOUS MATRIKS..... 7

PENGUMPULAN..... 13

KONSEP TRANSFORMASI

With the procedures for displaying output primitives and their attributes, we can create a variety of pictures and graphs. In many applications, there is also a need for altering or manipulating displays. Design applications and facility layouts are created by arranging the orientations and sizes of the component parts of the scene. And animations are produced by moving the “camera” or the objects in a scene along animation paths. Changes in orientation, size, and shape are accomplished with **geometric transformations** that alter the coordinate descriptions of objects. The basic geometric transformations are translation, rotation, and scaling. Other transformations that are often applied to objects include reflection and shear. We first discuss methods for performing geometric transformations and then consider how transformation functions can be incorporated into graphics packages.

TRANSLASI

Translation

A **translation** is applied to an object by repositioning it along a straight-line path from one coordinate location to another. We translate a two-dimensional point by adding **translation distances**, t_x and t_y , to the original coordinate position (x, y) to move the point to a new position (x', y') (Fig. 5-1).

$$x' = x + t_x, \quad y' = y + t_y \quad (5-1)$$

The translation distance pair (t_x, t_y) is called a **translation vector** or **shift vector**.

We can express the translation equations 5-1 as a single matrix equation by using column vectors to represent coordinate positions and the translation vector:

$$\mathbf{P} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \mathbf{P}' = \begin{bmatrix} x'_1 \\ x'_2 \end{bmatrix}, \quad \mathbf{T} = \begin{bmatrix} t_x \\ t_y \end{bmatrix} \quad (5-2)$$

This allows us to write the two-dimensional translation equations in the matrix form:

$$\mathbf{P}' = \mathbf{P} + \mathbf{T} \quad (5-3)$$

Sometimes matrix-transformation equations are expressed in terms of coordinate row vectors instead of column vectors. In this case, we would write the matrix representations as $\mathbf{P} = [x \ y]$ and $\mathbf{T} = [t_x \ t_y]$. Since the column-vector representation for a point is standard mathematical notation, and since many graphics packages, for example, GKS and PHIGS, also use the column-vector representation, we will follow this convention.

Translation is a *rigid-body transformation* that moves objects without defor-

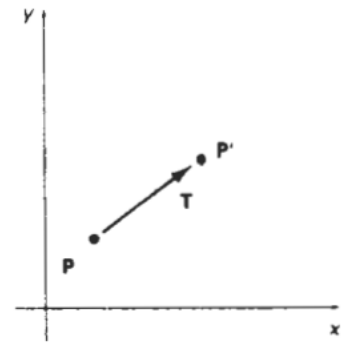


Figure 5-1
Translating a point from position $\mathbf{P}$ to position $\mathbf{P}'$ with translation vector $\mathbf{T}$.

Similar methods are used to translate curved objects. To change the position of a circle or ellipse, we translate the center coordinates and redraw the figure in the new location. We translate other curves (for example, splines) by displacing the coordinate positions defining the objects, then we reconstruct the curve paths using the translated coordinate points.

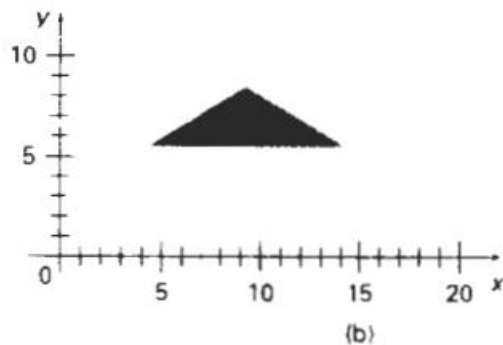
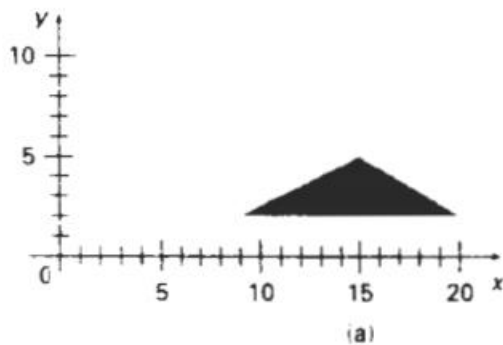


Figure 5-2
Moving a polygon from position (a) to position (b) with the translation vector $(-5.50, 3.75)$.

SCALING

A **scaling** transformation alters the size of an object. This operation can be carried out for polygons by multiplying the coordinate values (x , y) of each vertex by **scaling factors** s_x and s_y to produce the transformed coordinates (x' , y'):

$$x' = x \cdot s_x, \quad y' = y \cdot s_y \quad (5-10)$$

Scaling factor s_x scales objects in the x direction, while s_y scales in the y direction. The transformation equations 5-10 can also be written in the matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \quad (5-11)$$

or

$$\mathbf{P}' = \mathbf{S} \cdot \mathbf{P} \quad (5-12)$$

where $\mathbf{S}$ is the 2 by 2 scaling matrix in Eq. 5-11.

Any positive numeric values can be assigned to the scaling factors s_x and s_y . Values less than 1 reduce the size of objects; values greater than 1 produce an enlargement. Specifying a value of 1 for both s_x and s_y leaves the size of objects unchanged. When s_x and s_y are assigned the same value, a **uniform scaling** is pro-

duced that maintains relative object proportions. Unequal values for s_x and s_y result in a **differential scaling** that is often used in design applications, where pictures are constructed from a few basic shapes that can be adjusted by scaling and positioning transformations (Fig. 5-6).

Objects transformed with Eq. 5-11 are both scaled and repositioned. Scaling factors with values less than 1 move objects closer to the coordinate origin, while values greater than 1 move coordinate positions farther from the origin. Figure 5-7 illustrates scaling a line by assigning the value 0.5 to both s_x and s_y in Eq. 5-11. Both the line length and the distance from the origin are reduced by a factor of 1/2.

We can rewrite these scaling transformations to separate the multiplicative and additive terms:

$$\begin{aligned}x' &= x \cdot s_x + x_f(1 - s_x) \\y' &= y \cdot s_y + y_f(1 - s_y)\end{aligned}\tag{5-14}$$

where the additive terms $x_f(1 - s_x)$ and $y_f(1 - s_y)$ are constant for all points in the object.

ROTASI

Rotation

A two-dimensional **rotation** is applied to an object by repositioning it along a circular path in the xy plane. To generate a rotation, we specify a **rotation angle** θ and the position (x_r, y_r) of the **rotation point** (or **pivot point**) about which the object is to be rotated (Fig. 5-3). Positive values for the rotation angle define counterclockwise rotations about the pivot point, as in Fig. 5-3, and negative values rotate objects in the clockwise direction. This transformation can also be described as a rotation about a **rotation axis** that is perpendicular to the xy plane and passes through the pivot point.

We first determine the transformation equations for rotation of a point position **P** when the pivot point is at the coordinate origin. The angular and coordinate relationships of the original and transformed point positions are shown in Fig. 5-4. In this figure, r is the constant distance of the point from the origin, angle ϕ is the original angular position of the point from the horizontal, and θ is the rotation angle. Using standard trigonometric identities, we can express the transformed coordinates in terms of angles θ and ϕ as

$$\begin{aligned}x' &= r \cos (\phi + \theta) = r \cos \phi \cos \theta - r \sin \phi \sin \theta \\y' &= r \sin (\phi + \theta) = r \cos \phi \sin \theta + r \sin \phi \cos \theta\end{aligned}\tag{5-4}$$

The original coordinates of the point in polar coordinates are

$$x = r \cos \phi, \quad y = r \sin \phi \quad (5-5)$$

Substituting expressions 5-5 into 5-4, we obtain the transformation equations for rotating a point at position (x, y) through an angle θ about the origin:

$$\begin{aligned} x' &= x \cos \theta - y \sin \theta \\ y' &= x \sin \theta + y \cos \theta \end{aligned} \quad (5-6)$$

With the column-vector representations 5-2 for coordinate positions, we can write the rotation equations in the matrix form:

$$\mathbf{P}' = \mathbf{R} \cdot \mathbf{P} \quad (5-7)$$

where the rotation matrix is

$$\mathbf{R} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad (5-8)$$

$$[\sin \theta \quad \cos \theta]$$

When coordinate positions are represented as row vectors instead of column vectors, the matrix product in rotation equation 5-7 is transposed so that the transformed row coordinate vector $[x' \ y']$ is calculated as

$$\begin{aligned} \mathbf{P}'^T &= (\mathbf{R} \cdot \mathbf{P})^T \\ &= \mathbf{P}^T \cdot \mathbf{R}^T \end{aligned}$$

where $\mathbf{P}^T = [x \ y]$, and the transpose $\mathbf{R}^T$ of matrix $\mathbf{R}$ is obtained by interchanging rows and columns. For a rotation matrix, the transpose is obtained by simply changing the sign of the sine terms.

Rotation of a point about an arbitrary pivot position is illustrated in Fig. 5-5. Using the trigonometric relationships in this figure, we can generalize Eqs. 5-6 to obtain the transformation equations for rotation of a point about any specified rotation position (x_r, y_r) :

$$\begin{aligned}x' &= x_r + (x - x_r) \cos \theta - (y - y_r) \sin \theta \\y' &= y_r + (x - x_r) \sin \theta + (y - y_r) \cos \theta\end{aligned}\tag{5-9}$$

These general rotation equations differ from Eqs. 5-6 by the inclusion of additive terms, as well as the multiplicative factors on the coordinate values. Thus, the matrix expression 5-7 could be modified to include pivot coordinates by matrix addition of a column vector whose elements contain the additive (translational) terms in Eqs. 5-9. There are better ways, however, to formulate such matrix equations, and we discuss in Section 5-2 a more consistent scheme for representing the transformation equations.

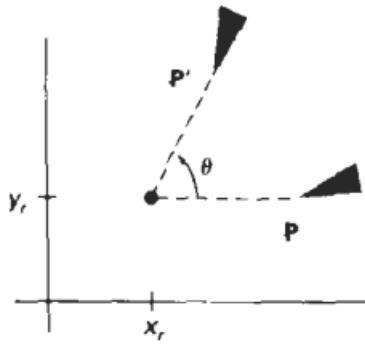


Figure 5-3
Rotation of an object through angle θ about the pivot point (x_r, y_r) .

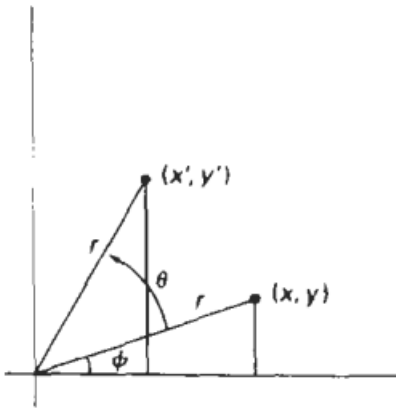


Figure 5-4
Rotation of a point from position (x, y) to position (x', y') through an angle θ relative to the coordinate origin. The original angular displacement of the point from the x axis is ϕ .

Section 5-1

Basic Transformations

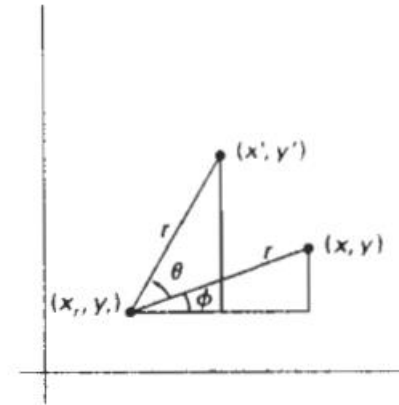


Figure 5-5
Rotating a point from position (x, y) to position (x', y') through an angle θ about rotation point (x_r, y_r) .

HOMOGENEOUS MATRIKS

Many graphics applications involve sequences of geometric transformations. An animation, for example, might require an object to be translated and rotated at each increment of the motion. In design and picture construction applications,

we perform translations, rotations, and scalings to fit the picture components into their proper positions. Here we consider how the matrix representations discussed in the previous sections can be reformulated so that such transformation sequences can be efficiently processed.

We have seen in Section 5-1 that each of the basic transformations can be expressed in the general matrix form

$$\mathbf{P}' = \mathbf{M}_1 \cdot \mathbf{P} + \mathbf{M}_2 \quad (5-15)$$

with coordinate positions $\mathbf{P}$ and $\mathbf{P}'$ represented as column vectors. Matrix $\mathbf{M}_1$ is a 2 by 2 array containing multiplicative factors, and $\mathbf{M}_2$ is a two-element column matrix containing translational terms. For translation, $\mathbf{M}_1$ is the identity matrix. For rotation or scaling, $\mathbf{M}_2$ contains the translational terms associated with the pivot point or scaling fixed point. To produce a sequence of transformations with these equations, such as scaling followed by rotation then translation, we must calculate the transformed coordinates one step at a time. First, coordinate positions are scaled, then these scaled coordinates are rotated, and finally the rotated coordinates are translated. A more efficient approach would be to combine the transformations so that the final coordinate positions are obtained directly from the initial coordinates, thereby eliminating the calculation of intermediate coordinate values. To be able to do this, we need to reformulate Eq. 5-15 to eliminate the matrix addition associated with the translation terms in $\mathbf{M}_2$.

We can combine the multiplicative and translational terms for two-dimensional geometric transformations into a single matrix representation by expanding the 2 by 2 matrix representations to 3 by 3 matrices. This allows us to express all transformation equations as matrix multiplications, providing that we also expand the matrix representations for coordinate positions. To express any two-dimensional transformation as a matrix multiplication, we represent each Cartesian coordinate position (x, y) with the **homogeneous coordinate** triple (x_h, y_h, h) , where

$$x = \frac{x_h}{h}, \quad y = \frac{y_h}{h} \quad (5-16)$$

Thus, a general homogeneous coordinate representation can also be written as $(h \cdot x, h \cdot y, h)$. For two-dimensional geometric transformations, we can choose the homogeneous parameter h to be any nonzero value. Thus, there is an infinite number of equivalent homogeneous representations for each coordinate point (x, y) . A convenient choice is simply to set $h = 1$. Each two-dimensional position is then represented with homogeneous coordinates $(x, y, 1)$. Other values for parameter h are needed, for example, in matrix formulations of three-dimensional viewing transformations.

Thus, a general homogeneous coordinate representation can also be written as $(h \cdot x, h \cdot y, h)$. For two-dimensional geometric transformations, we can choose the homogeneous parameter h to be any nonzero value. Thus, there is an infinite number of equivalent homogeneous representations for each coordinate point (x, y) . A convenient choice is simply to set $h = 1$. Each two-dimensional position is then represented with homogeneous coordinates $(x, y, 1)$. Other values for parameter h are needed, for example, in matrix formulations of three-dimensional viewing transformations.

The term *homogeneous coordinates* is used in mathematics to refer to the effect of this representation on Cartesian equations. When a Cartesian point (x, y) is converted to a homogeneous representation (x_h, y_h, h) , equations containing x and y , such as $f(x, y) = 0$, become homogeneous equations in the three parameters x_h, y_h , and h . This just means that if each of the three parameters is replaced by any value v times that parameter, the value v can be factored out of the equations.

represented with three-element column vectors, and transformation operations are written as 3 by 3 matrices. For translation, we have

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (5-17)$$

which we can write in the abbreviated form

$$\mathbf{P}' = \mathbf{T}(t_x, t_y) \cdot \mathbf{P} \quad (5-18)$$

Similarly, rotation transformation equations about the coordinate origin are now written as

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (5-19)$$

or as

$$\mathbf{P}' = \mathbf{R}(\theta) \cdot \mathbf{P} \quad (5-20)$$

Finally, a scaling transformation relative to the coordinate origin is now expressed as the matrix multiplication

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (5-21)$$

or

$$\mathbf{P}' = \mathbf{S}(s_x, s_y) \cdot \mathbf{P} \quad (5-22)$$

```

#include <math.h>
#include "graphics.h"

typedef float Matrix3x3[3][3];
Matrix3x3 theMatrix;

void matrix3x3SetIdentity (Matrix3x3 m)
{
    int i,j;

    for (i=0; i<3; i++) for (j=0; j<3; j++) m[i][j] = (i == j);
}

/* Multiplies matrix a times b, putting result in b */
void matrix3x3PreMultiply (Matrix3x3 a, Matrix3x3 b)
{
    int r,c;
    Matrix3x3 tmp;

    for (r = 0; r < 3; r++)
        for (c = 0; c < 3; c++)
            tmp[r][c] =
                a[r][0]*b[0][c] + a[r][1]*b[1][c] + a[r][2]*b[2][c];

    for (r = 0; r < 3; r++)
        for (c = 0; c < 3; c++)
            b[r][c] = tmp[r][c];
}

void translate2 (int tx, int ty)
{
    Matrix3x3 m;

    matrix3x3SetIdentity (m);
    m[0][2] = tx;
    m[1][2] = ty;
    matrix3x3PreMultiply (m, theMatrix);
}

```



```

void scale2 (float sx, float sy, wcPt2 refpt)
{
    Matrix3x3 m;

    matrix3x3SetIdentity (m);
    m[0][0] = sx;
    m[0][2] = (1 - sx) * refpt.x;
    m[1][1] = sy;
    m[1][2] = (1 - sy) * refpt.y;
    matrix3x3PreMultiply (m, theMatrix);
}

void rotate2 (float a, wcPt2 refPt)
{
    Matrix3x3 m;

    matrix3x3SetIdentity (m);
    a = pToRadians (a);
    m[0][0] = cosf (a);
    m[0][1] = -sinf (a);
    m[0][2] = refPt.x * (1 - cosf (a)) + refPt.y * sinf (a);
    m[1][0] = sinf (a);
    m[1][1] = cosf (a);
    m[1][2] = refPt.y * (1 - cosf (a)) - refPt.x * sinf (a);
    matrix3x3PreMultiply (m, theMatrix);
}

void transformPoints2 (int npts, wcPt2 *pts)
{
    int k;
    float tmp;

    for (k = 0; k < npts; k++) {
        tmp = theMatrix[0][0] * pts[k].x + theMatrix[0][1] *
            pts[k].y + theMatrix[0][2];
        pts[k].y = theMatrix[1][0] * pts[k].x + theMatrix[1][1] *
            pts[k].y + theMatrix[1][2];
        pts[k].x = tmp;
    }
}

```



```

void main (int argc, char ** argv)
{
    wcPt2 pts[3] = { 50.0, 50.0, 150.0, 50.0, 100.0, 150.0};
    wcPt2 refPt = {100.0, 100.0};
    long windowID = openGraphics (*argv, 200, 350);

    setBackground (WHITE);
    setColor (BLUE);
    pFillArea (3, pts);
    matrix3x3SetIdentity (theMatrix);
    scale2 (0.5, 0.5, refPt);
    rotate2 (90.0, refPt);
    translate2 (0, 150);
    transformPoints2 (3, pts),
    pFillArea (3,pts);
    sleep (10);
    closeGraphics (windowID);
}

```

TASK 0: ANALISA TRANSFORMASI

Tugas Task 0: Buat Lesson Learnt untuk pertanyaan-pertanyaan berikut

1. Bedah code Transformasi.cs pada file template yang diberikan oleh Dosen
2. Adaptasi Transformasi.cs pada code praktikum modul 3 anda
3. Apa yang anda pahami tentang homogenous matriks / coordinates
4. Apa yang anda pahami dari fungsi transformPoints
5. Mana yang lebih efisien melakukan transformPoints untuk semua list vector sebuah bentuk, atau cukup melakukan transformPoints, titik-titik pembangun bentuk?

Lesson Learnt

Menganalisa Transformasi dalam code yang di berikan di modul 4 ada beberapa yang code

1. Bedah Code Transformasi.cs

- **Bagian Matrix Operations**
 - Matrix3x3Identity → bikin matriks identitas (tidak mengubah apapun, titik tetap di posisi awal).
 - Matrix3x3Summation & Matrix3x3Subtraction → menjumlahkan/ mengurangi dua matriks.
 - Matrix3x3Multiplication → melakukan perkalian dua matriks 3x3 (inti dari semua transformasi).
- **Bagian Geometric Transformations**
 - GetTransformPoint → menerapkan matriks transformasi ke setiap titik dalam list res, lalu mengembalikan list titik baru.
 - Translation, Scaling, RotationClockwise, RotationCounterClockwise, Shearing, ReflectionToX/Y/Origin → semua fungsi ini membentuk matriks transformasi spesifik, lalu mengalikan matriks hasilnya dengan matriks transformasi yang sudah ada.
 - Juga bahwa beberapa transformasi mempertimbangkan **pivot point** (coord) — ini penting kalau kamu mau rotasi atau skala terhadap pusat objek, bukan terhadap (0,0).

2. Adaptasi Transformasi.cs pada code praktikum modul 3

untuk adaptasi kita jadi mudah, cukup hanya memanggil fungsi tersebut tanpa kita harus membuat manual algoritma nya / rumus nya

3. Pemahaman tentang Homogenous Matrix / Coordinates

- Matriks homogen dipakai untuk menyatukan translasi, rotasi, scaling, dan shearing ke dalam **satu bentuk matematis** (3x3 matrix).
- Alih-alih mengubah x dan y secara manual, kamu cukup mengalikan vektor $[x, y, 1]$ dengan matriks 3x3.
- Ini penting di game dev karena:
 - Bisa *combine* beberapa transformasi (scaling lalu rotasi lalu translasi) jadi satu matriks komposit.
 - Efisien: hanya satu kali perkalian matriks untuk setiap titik.
 - Konsisten: semua transformasi mengikuti urutan yang jelas (*order matters*).

4. Pemahaman tentang fungsi transformPoints (atau GetTransformPoint)

Fungsi ini adalah “penerjemah” dari matriks transformasi ke posisi titik nyata.

- Dia iterasi semua Vector2 pada list yang diberikan.
- Menghitung x' dan y' hasil transformasi (dengan operasi dot product terhadap baris matriks).
- Mengembalikan list baru dengan posisi yang sudah diubah.

5. Efisiensi Transformasi: Semua List vs Titik Pembangun Bentuk

- Jika punya **bentuk kompleks** dengan ratusan titik (misalnya mesh), transformasi seluruh list bisa jadi mahal ($O(n)$ tiap frame).
- Tapi kalau bentuknya hanya didefinisikan oleh sedikit titik kunci (misalnya persegi cukup 4 titik, lalu engine yang menggambar sisanya), lebih efisien transformasi titik pembangun saja.

Jadi yang lebih efisien:

Transformasi titik-titik pembangun bentuk, lalu biarkan engine menggambar ulang bentuk berdasarkan titik yang sudah ditransformasi.

Transformasi seluruh list vector kalau data terlalu banyak — itu boros CPU dan bisa bikin framerate drop.

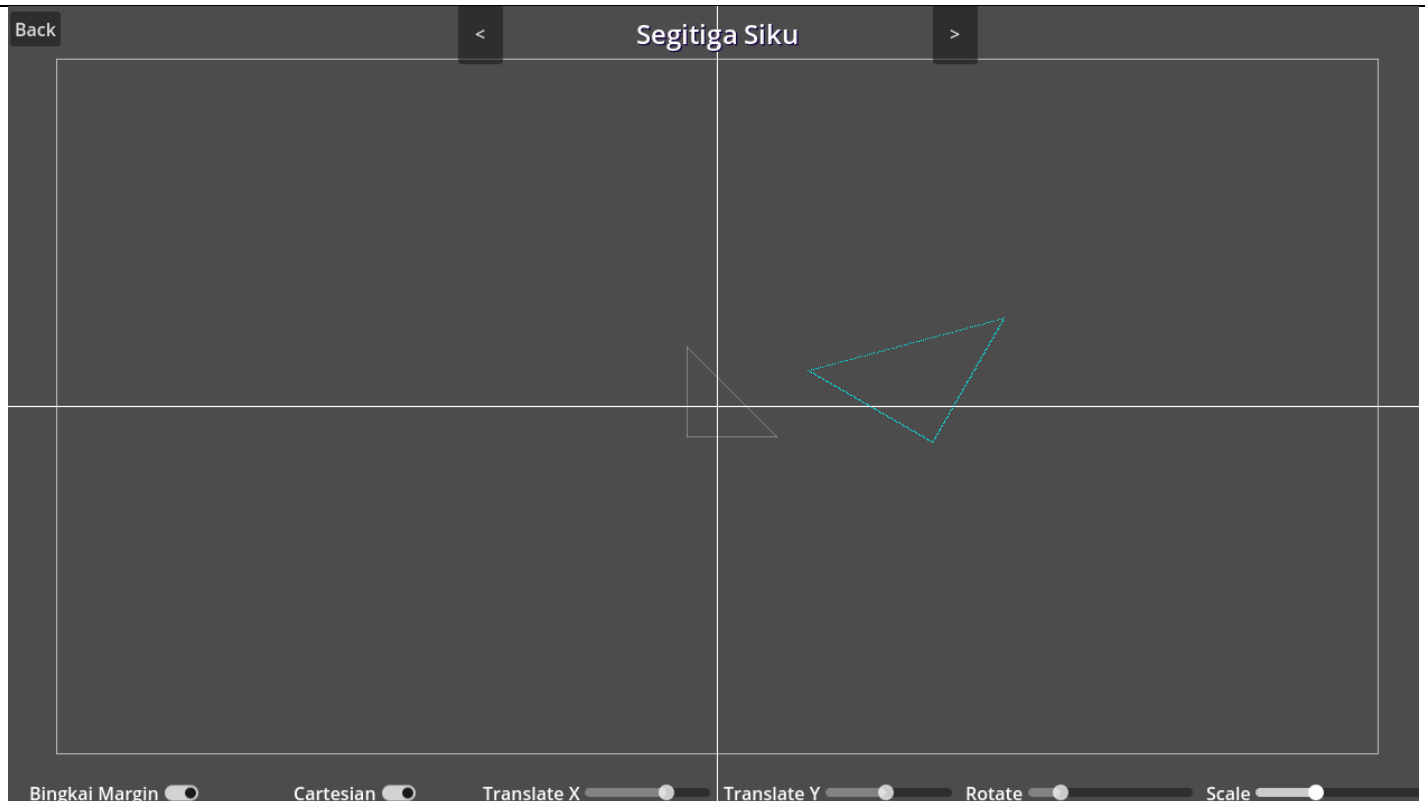
TASK I: EKSPLORASI TRANSFORMASI

Tugas Task I (Karya I)

1. Copy Karya 3 pada Modul 3 ke Karya I Pada Modul 4

2. Persentasikan Bentuk Bentuk Dasar Transformasi (Translasi, Scaling, Rotasi, Translasi + Scaling, Scaling+Translasi)

Lesson Learnt



- **Penerapan Matriks 3x3:** Saya belajar bahwa semua transformasi — translasi, scaling, dan rotasi — bisa diimplementasikan dengan menggunakan matriks 3x3. Pendekatan ini memudahkan kombinasi transformasi sekaligus menjaga konsistensi urutan operasi.
- **Pentingnya Pivot Point:** Scaling dan rotasi sebaiknya dilakukan terhadap pivot (pusat bentuk). Jika pivot tidak diatur dengan benar, bentuk dapat bergeser dari posisi aslinya sehingga hasil transformasi menjadi tidak stabil.
- **Urutan Transformasi:** Urutan sangat berpengaruh terhadap hasil akhir. Melakukan scaling dan rotasi terlebih dahulu di pivot sebelum translasi membuat pergerakan bentuk lebih natural dan smooth.
- **Eksperimen Interaktif:** Dengan adanya slider dan tombol yang bisa diubah secara real-time, saya bisa melihat langsung efek kombinasi transformasi. Ini membantu memahami bagaimana setiap parameter berpengaruh terhadap bentuk.
- **Visualisasi Dua Bentuk:** Menampilkan bentuk asli dan hasil transformasi secara bersamaan memberi perbandingan visual yang jelas, sehingga konsep transformasi lebih mudah dipahami secara intuitif.
- **Struktur Kode Modular:** Memisahkan logika untuk tiap transformasi membuat kode lebih mudah dipahami, dikembangkan, dan di-debug saat terjadi kesalahan.
- **Implementasi Semua bentuk menggunakan next/prev :** dalam program terlihat bahwa ada next prev disana yang ditujukan untuk menguji semua bentuk dalam melakukan transformasi

Kesimpulan :

- Translasi, scaling, dan rotasi memiliki fungsi berbeda namun saling melengkapi dalam membentuk transformasi kompleks.
- Urutan transformasi menentukan hasil akhir dan dapat menghasilkan efek visual yang sangat berbeda.

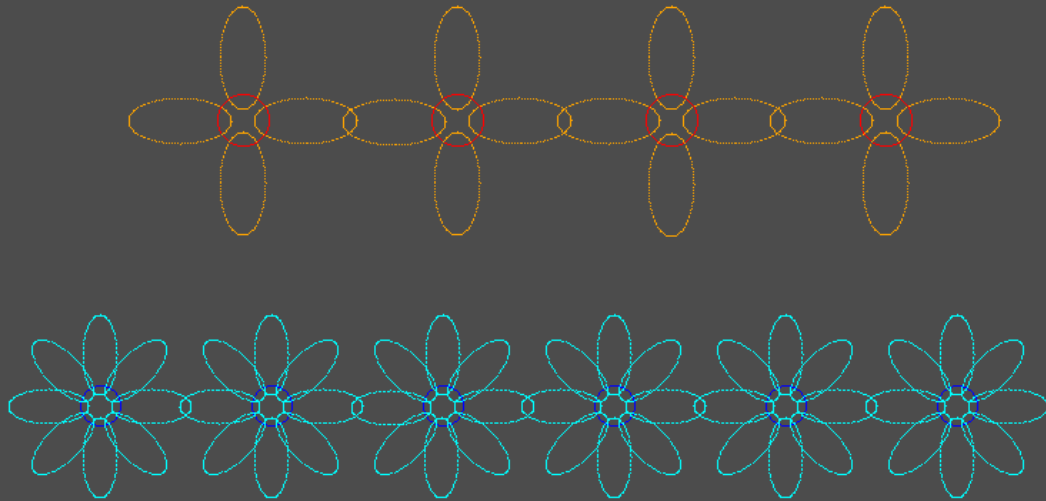
- Penggunaan matriks dan pivot point adalah kunci utama untuk mendapatkan hasil transformasi yang halus dan akurat.
- Melalui visualisasi dan interaksi langsung di *Karya1*, saya bisa memahami konsep transformasi grafika komputer secara lebih praktis dan mendalam.

TASK 2: KARYA 2D BUNGA TRANSFORMASI

Tugas Task 2 Karya 2

1. Buatlah sebuah bunga yang terdiri dari lingkaran pusat dan 4 kelopak ellips (syarat buat satu kelopak terlebih dahulu lalu gunakan transformasi rotate untuk membuat kelopak lainnya)
2. Buatlah sebuah bunga yang terdiri dari lingkaran pusat dan 8 kelopak ellips (syarat buat satu kelopak terlebih dahulu lalu gunakan transformasi rotate untuk membuat kelopak lainnya)
3. Setiap bunga dibuat menjadi fungsi parametrik atau object (OOP) untuk memudahkan penduplikasian, duplikasikan bunga tersebut sebanyak 4 untuk bunga tipe 1, dan 6 untuk bunga tipe 2.

Lesson Learnt



1. Pemodelan OOP Membuat Objek Geometri Lebih Fleksibel

Dalam proyek ini saya belajar bahwa dengan membuat kelas Flower, BentukDasar, dan Transformasi, saya bisa menggambar bentuk-bentuk kompleks tanpa harus menulis ulang logika.

Saya tidak lagi menggambar secara manual, tetapi membuat **objek matematis** yang bisa saya duplikasi, ubah posisi, dan rotasi sesuai kebutuhan.

Jadi, kalau saya ingin membuat bunga jenis baru, saya cukup mengganti parameter seperti:

```
new Flower(center, radius, 8, 30, 10, Colors.Cyan, Colors.Blue, _transform);
```

Tanpa harus mengubah struktur programnya lagi.

2. Transformasi Matriks Adalah Dasar dari Gerakan Geometri

Saya menyadari bahwa semua rotasi dan perpindahan yang terjadi pada kelopak bunga bisa dijelaskan lewat **matriks transformasi 3x3**.

Dengan matriks ini, saya bisa:

1. Memutar posisi pusat kelopak mengelilingi pusat bunga.
2. Memutar bentuk elips itu sendiri terhadap pusat kelopaknya.

Dari sini saya memahami bahwa di dunia grafika komputer, semua operasi seperti rotasi, translasi, scaling, bahkan refleksi, bisa dilakukan hanya dengan operasi matriks.

3. Koordinat Dunia dan Koordinat Layar Itu Berbeda

Fungsi ScreenUtils.ToScreenCoordinate() membuat saya sadar bahwa sistem koordinat dunia dan layar tidak sama. Kadang sumbu **Y terbalik** (naik di dunia, tapi turun di layar).

Saya jadi belajar kalau ingin bentuk tampil di tempat yang tepat, saya harus memahami arah sistem koordinat yang sedang digunakan.

Jadi ketika gambar tampak terbalik atau melenceng, masalahnya sering bukan di rumus, tapi di **konversi koordinat**.

4. Orientasi Elips Butuh Offset Rotasi Tambahan

Saya juga belajar bahwa elips default dari persamaan $x = r_x * \cos(t)$ dan $y = r_y * \sin(t)$ itu posisinya **horizontal**.

Agar elips terlihat **berdiri**, saya perlu menambahkan offset rotasi +90f.

Hal kecil ini ternyata sangat penting, karena tanpa offset, bentuk yang sama bisa tampak miring atau terbalik.

5. Jarak antara Lingkaran dan Kelopak Ditentukan Secara Matematis

Awalnya saya bingung kenapa jarak antara lingkaran pusat dan elips tidak berubah meski saya ubah angka.

Setelah memahami urutannya, saya sadar jarak itu diatur dari posisi awal initialPetalCenter.

Perubahan jarak berarti mengubah nilai pada baris ini:

```
Vector2 initialPetalCenter = new Vector2(Center.X, Center.Y + (CenterRadius + PetalRy + 10));
```

Saya belajar bahwa **mengatur jarak** tidak berarti memindahkan gambar secara manual, tapi mengubah posisi awal sebelum rotasi dilakukan.

6. Membangun Secara Bertahap Membantu Saya Memahami Lebih Dalam

Saya tidak langsung membuat bunga lengkap, tapi melakukannya tahap demi tahap:

1. Gambar satu lingkaran pusat.
2. Tambahkan satu kelopak.
3. Gunakan rotasi untuk menambah kelopak lain.
4. Gandakan bunganya.

Pendekatan bertahap seperti ini membantu saya memahami hubungan antar langkah dan lebih mudah mencari kesalahan saat hasilnya belum sesuai.

7. Visualisasi Jadi Bukti Nyata dari Transformasi

Melihat bentuk bunga terbentuk di layar membuat saya sadar bahwa setiap titik yang diputar dan ditranslasikan benar-benar mengikuti logika matematika.

Saya merasa seperti melihat konsep *Model-View Transformation* dalam grafika komputer bekerja secara nyata.

Kesimpulan

Dari proyek ini saya belajar bahwa:

- Saya bisa menggunakan **OOP** untuk mengatur logika visual secara modular.
- **Transformasi matriks** adalah kunci utama di balik semua gerakan geometri di dunia grafika.
- **Koordinat dunia dan layar** punya arah berbeda, jadi saya harus selalu berhati-hati saat menggambar.
- **Pivot dan orientasi lokal** menentukan bagaimana suatu bentuk berputar.
- **Jarak antar-objek** bisa diatur hanya dengan mengubah posisi awal sebelum transformasi.

Secara keseluruhan, saya merasa proyek ini bukan hanya tentang menggambar bunga, tetapi tentang **memahami cara kerja sistem grafika komputer dari dalam** — dari matematika hingga visualisasi.

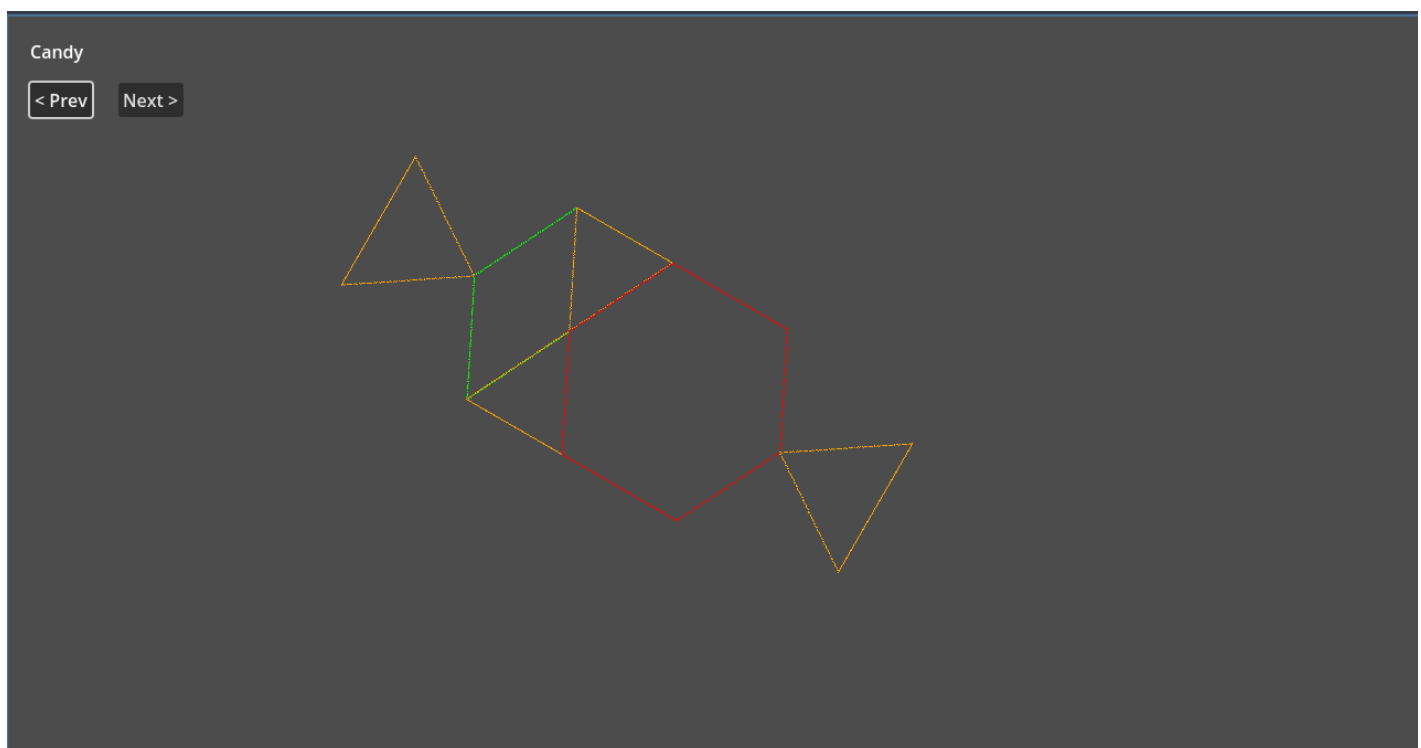
Saya jadi jauh lebih paham bagaimana rotasi, posisi, dan bentuk bisa diatur dengan presisi menggunakan logika transformasi.

TASK 3: KARYA 2D BUNGA TRANSFORMASI ANIMASI

Tugas Task 3 Karya 3 (Animasinya dapat dilanjutkan minggu depan)

1. Semenarik mungkin tuangkan list ide-ide animasi untuk bunga tipe 1 dan tipe 2
2. Urutkan ide tersebut dari yang paling sederhana hingga yang paling kompleks. lalu pilih 2 ide animasi sederhana dan animasi kompleks.
3. Buatlah Design atau storyboard untuk animasi sederhana setiap bunga tipe 1 dan tipe 2 untuk (misal, untuk frame 1-60, bunga translasi +x dengan jarak 10 lalu berotasi 360 derajat lalu kembali ke tempat semula, dst).
4. Buatlah Design atau storyboard untuk animasi kompleks setiap bunga tipe 1 dan tipe 2 untuk (misal, untuk frame 1-60, bunga translasi +x dengan jarak 10 lalu berotasi 360 derajat lalu kembali ke tempat semula, dst).
5. Pada karya 3 implementasikan ide tersebut (untuk minggu depan).

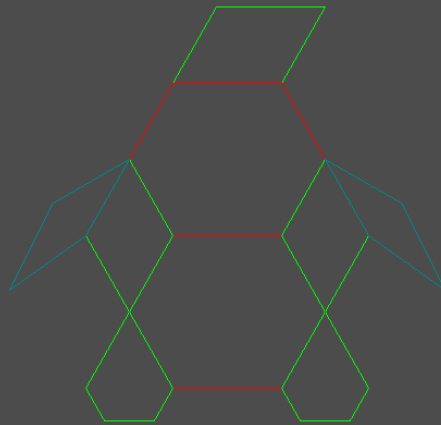
Lesson Learnt



Penguin

< Prev

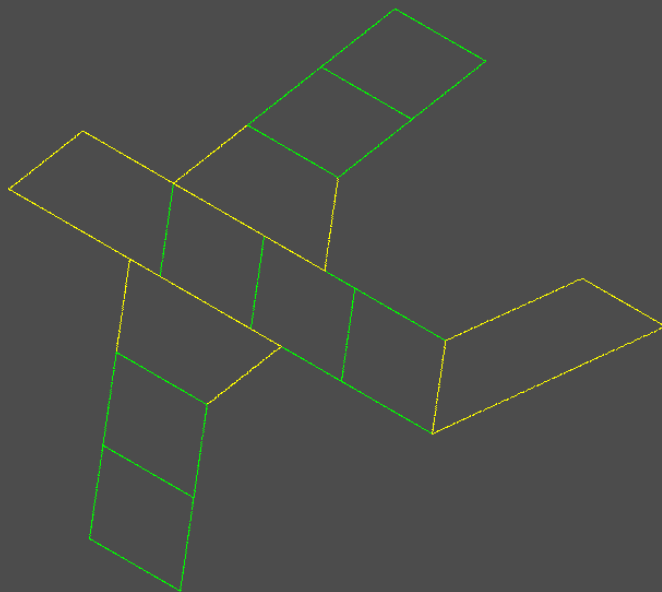
Next >



Plane

< Prev

Next >



Saya membuat proyek *Karya3* untuk memperdalam pemahaman saya tentang **transformasi geometrik 2D**

menggunakan pendekatan **OOP dan parametrik** di Godot C#.

Saya ingin membuktikan bahwa bentuk-bentuk kompleks seperti permen, penguin, dan pesawat bisa dibangun dari gabungan bentuk dasar dengan logika transformasi yang sistematis.

1. Pemahaman Konseptual

Saya belajar bahwa:

- **Setiap bentuk kompleks** (seperti Candy, Penguin, atau Plane) bisa dipecah menjadi bentuk-bentuk dasar seperti **segitiga, trapesium, jajar genjang, dan belah ketupat**.
- **Transformasi 2D** (rotasi, translasi, refleksi, dan skala) tidak hanya mengubah posisi, tapi juga menjadi *alat kreatif* untuk menyusun bentuk yang simetris, dinamis, dan artistik.
- Matriks transformasi (baik 3×3 maupun Matrix4x4) memungkinkan bentuk diputar dan digeser tanpa harus menulis ulang koordinat satu per satu.

2. Proses dan Strategi yang Saya Gunakan

Saya menggunakan pendekatan berikut:

1. **Membuat fungsi ToScreenPolygon()**, agar semua titik hasil transformasi otomatis dikonversi ke sistem layar (screen coordinate).
2. **Menggunakan struktur GambarBentuk (name, draw)** untuk membuat sistem navigasi antar karya dengan tombol Next/Prev secara modular.
3. **Memisahkan setiap bentuk ke fungsi tersendiri** (DrawCandy(), DrawPenguin(), DrawPlane()), agar setiap karya punya logika visual yang independen dan bisa dikembangkan terpisah.
4. **Menguji transformasi secara bertahap** — saya tidak langsung menggambar keseluruhan, tapi mulai dari bentuk dasar lalu menambahkan elemen satu per satu (seperti segitiga kiri → hexagon → segitiga kanan pada Candy).

3. Tantangan yang Saya Hadapi

Beberapa hal yang saya sadari:

- Saat awal, posisi bentuk sering **terbalik atau keluar dari layar** karena saya belum memahami urutan transformasi (rotasi dulu atau translasi dulu).
- **Koordinat pivot** sangat berpengaruh. Jika salah posisi pivot, bentuk bisa berputar “liar” atau tidak seimbang.
- **Perbedaan sistem koordinat Godot dan kartesius matematis** membuat saya harus menambahkan refleksi sumbu X agar orientasi tetap sesuai ekspektasi visual.
- Bentuk yang simetris (seperti permen) menuntut **presisi dalam titik awal dan sudut rotasi** agar kedua sisi tampak seimbang.

4. Pembelajaran Teknis yang Saya Dapat

- Saya memahami bahwa **komposisi transformasi** harus berurutan:
Refleksi → Rotasi → Skala → Translasi
- **Matrix3x3Identity()** menjadi dasar penting agar setiap gambar dimulai dari sistem koordinat netral.
- **Penggunaan warna berbeda per bentuk** membantu saya melacak apakah posisi tiap bagian sudah benar.
- **Abstraksi kelas BentukDasar dan Transformasi** membuat saya tidak perlu menulis ulang kode titik koordinat, cukup panggil fungsi siap pakai.

5. Refleksi Pribadi

Saya belajar untuk:

- Tidak langsung fokus ke hasil visual, tapi memahami **alur logika transformasi** di baliknya.

- Menyadari bahwa “gambar” bukan sekadar hasil, tapi juga ekspresi dari urutan transformasi dan koordinat.
- Menemukan keseimbangan antara **kreativitas visual** dan **presisi matematis**.
- Lebih sabar saat debug bentuk yang salah posisi — karena kesalahan kecil pada urutan transformasi bisa mengacaukan semuanya.

PENGUMPULAN

Ikuti Format yang diberikan di Google Classroom.